

b) Investigación:

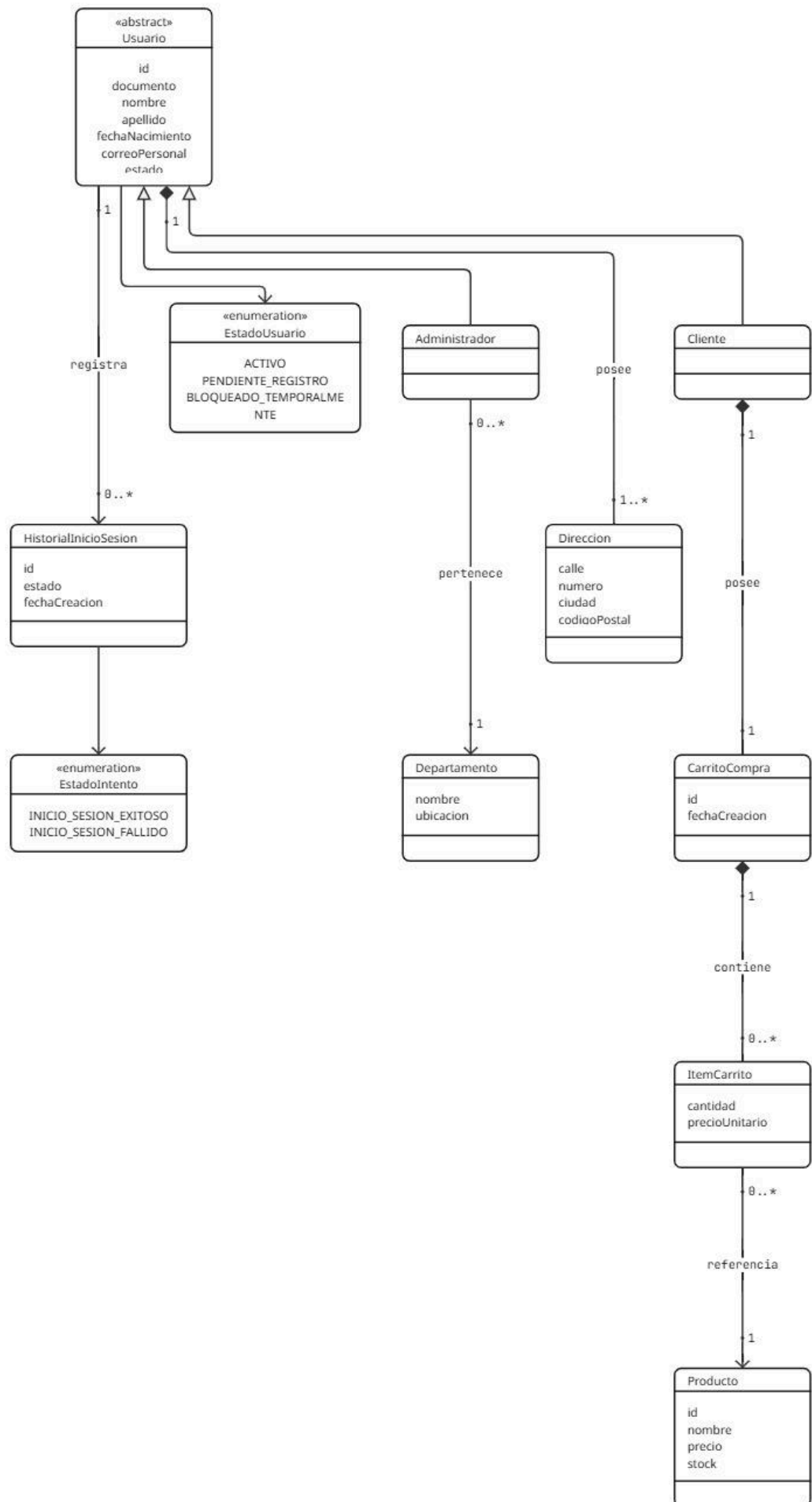
- Análisis y Diseño Orientado a Objetos:

¿Cuál es la diferencia entre el Diagrama de Clases de Análisis y el Diagrama de Clases de Diseño?

La diferencia principal es el nivel de detalle y el momento del desarrollo en el que se realiza cada uno. El Diagrama de Clases de **Análisis** se realiza durante la etapa de análisis. Su objetivo es representar qué elementos existen en el sistema y cómo se relacionan, pero sin preocuparse todavía demasiado por cómo se implementarán.

El Diagrama de Clases de **Diseño** se realiza durante la etapa de diseño. Parte del modelo de análisis y lo transforma en una estructura mucho más cercana a la implementación del software.

- Realizar el Diagrama de Clases de Análisis del Ejercicio “a”.



Diagramas de secuencia:

Un Diagrama de Secuencia es un diagrama UML que muestra cómo interactúan distintos objetos o clases a lo largo del tiempo.

Sirve para representar qué mensajes se envían entre ellos y en qué orden para realizar una determinada operación.

- Elementos principales
 - Actor: persona o sistema que inicia una acción.
 - Objeto o clase: participa en el proceso.
 - Línea de vida: línea vertical que representa el tiempo de existencia del objeto.
 - Mensajes: flechas entre los objetos que representan llamadas a métodos.
 - Retorno: indica la respuesta de una operación.
 - Activación: muestra el momento en que un objeto está realizando una tarea.
- Diagrama de secuencia de Diseño vs Diagrama de Secuencia de Análisis

El Diagrama de Secuencia de Análisis muestra el funcionamiento de forma general. Se concentra en qué debe hacer el sistema, sin entrar demasiado en cómo está programado.

El Diagrama de Secuencia de Diseño es más detallado y muestra cómo se implementará realmente, incluyendo las clases, métodos y capas del sistema.

- Ejemplo: ABM de Nacionalidad

Supongamos una entidad:

Nacionalidad

- nombre

Para realizar el ABM se pueden utilizar tres clases:

- NacionalidadController: recibe la solicitud.
- NacionalidadService: contiene la lógica de la operación.
- NacionalidadRepository: accede a la base de datos.

Por ejemplo, para crear una nacionalidad, la secuencia sería:

Usuario → Controller → Service → Repository → Base de Datos

Los mensajes podrían ser:

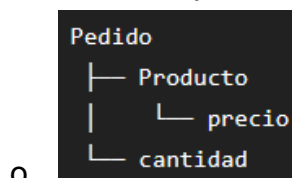
1. Usuario → Controller: crearNacionalidad(nombre)
2. Controller → Service: crearNacionalidad(nombre)

3. Service → Repository: guardar(nacionalidad)
4. Repository → Base de Datos: guarda la nacionalidad.
5. El resultado vuelve desde Repository → Service → Controller → Usuario.

Para buscar, modificar o eliminar una nacionalidad se sigue la misma estructura, cambiando el método correspondiente.

Así, el Controller recibe la petición, el Service procesa la lógica y el Repository realiza el acceso a los datos.

- Realizar una explicación de cada uno de los Patrones GRAPS. Además, investiga sobre el Patrón Modelo Vista Controlador (MVC). Explicar la vinculación que existe entre los patrones GRASP y el patrón MVC.
- Patrón GRASP.
 - GRASP reúne 9 patrones para decidir cómo asignar responsabilidades a clases y objetos en el diseño orientado a objetos.
 - Pregunta qué Responde: ¿Qué objeto o clase debería ser responsable de realizar determinada tarea?
 - **Patron “Experto en información”**
 - o La clase que conoce los datos debería encargarse de procesarlos.



o

```

class Pedido:
    def calcular_total(self):
        ...
  
```

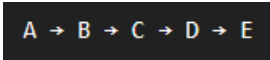
o

- **Patron “Creator”**
 - o Determina qué clase debería ser responsable de crear una instancia de otra clase.
 - o Una clase puede ser un buen creador de otra cuando, por ejemplo
 - § contiene objetos de esa clase;
 - § los agrupa;
 - § los utiliza mucho;
 - § posee los datos necesarios para inicializarlos

- **Patron “Controlador”**

- o Asigna a un objeto la responsabilidad de recibir y coordinar las solicitudes provenientes de una interfaz externa
 - § UI
 - § Petición HTTP
 - § Acción del usuario.
- o El Controller coordina; no debería concentrar toda la lógica de negocio.

- **Patron “Bajo Acoplamiento”**

- o Busca que las clases tengan la menor cantidad posible de dependencias innecesarias entre sí.
 - § 
 - § Un cambio en E podría provocar modificaciones en muchas clases.

- **Patrón “Alta cohesión”**

- o Una clase debería tener responsabilidades relacionadas entre sí y no convertirse en una clase que hace absolutamente todo.
 - § Lo que NO se recomienda

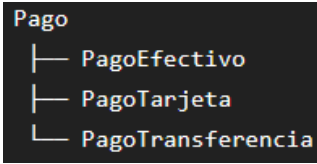
```
class Sistema:  
    crear_usuario()  
    enviar_email()  
    calcular_impuesto()
```

- § Es mejor Distribuir tales operaciones

```
UsuarioService  
EmailService  
ImpuestoService
```

- **Patrón “Polimorfismo”**

- o Cuando el comportamiento cambia dependiendo del tipo de objeto, se utiliza polimorfismo para que cada tipo implemente su comportamiento.

§ 

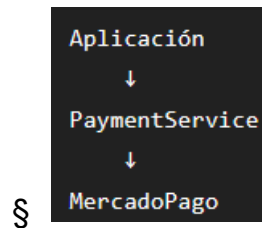
o .

- **Patrón “Fabricación pura”**

o .

- **Patrón “Indirección”**

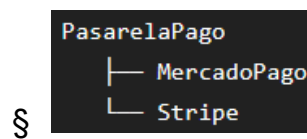
- o Se introduce un intermediario entre dos componentes para evitar que estén directamente acoplados.



o .

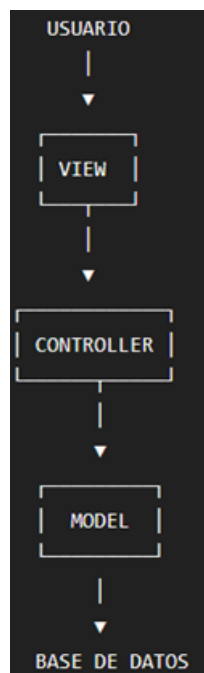
- Patrón Variaciones protegidas

- o Busca proteger al sistema de aquellas partes que sabemos que podrían cambiar.



Patrón MVC

- Es un patrón que busca principalmente separar responsabilidades, especialmente las relacionadas con la interfaz de usuario y la lógica de la aplicación.



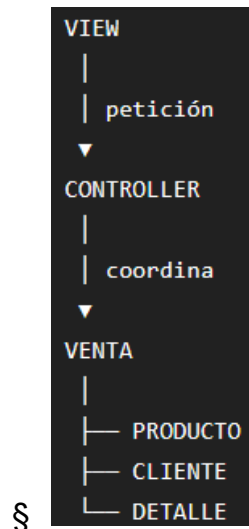
- .

¿Cómo se relacionan GRASP y MVC?

- MVC organiza componentes de una aplicación, especialmente alrededor de la interacción con el usuario.
- GRASP ayuda a decidir dónde colocar las responsabilidades dentro de esos componentes y clases.

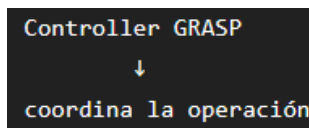
- Supongamos una aplicación de ventas.

- o MVC:

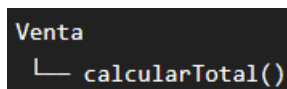


- o GRASP

- § Patron Controlador



- § Information Expert



1. ¿Qué es Spring Boot? ¿Cuál es su sitio web oficial? ¿Cómo se accede a la documentación?

Spring Boot es un framework basado en Spring que facilita el desarrollo de aplicaciones Java, reduciendo la cantidad de configuración necesaria. Permite crear aplicaciones independientes, utilizar servidores web embebidos y configurar automáticamente muchos componentes del proyecto.

Sitio web oficial: <https://spring.io/projects/spring-boot/>

Documentación oficial: <https://docs.spring.io/spring-boot/>

La documentación se puede consultar directamente desde el sitio oficial de documentación de Spring Boot.

2. ¿Qué es Spring Tools? ¿Cuál es su sitio web oficial?

Spring Tools es un conjunto de herramientas para desarrollar aplicaciones utilizando Spring Framework y Spring Boot. Proporciona funcionalidades como autocompletado, navegación del código, reconocimiento de componentes Spring e integración con Spring Initializr.

Actualmente está disponible para diferentes entornos de desarrollo como Visual Studio Code, Eclipse y otros.

Sitio web oficial: <https://spring.io/tools/>

3. ¿Qué es Spring Initializr? ¿Cuál es su sitio web oficial?

Spring Initializr es una herramienta que permite generar rápidamente la estructura inicial de un proyecto Spring Boot.

El usuario puede seleccionar el lenguaje, versión de Spring Boot, sistema de construcción como Maven o Gradle, versión de Java, dependencias necesarias y tipo de empaquetado.

Luego genera un proyecto que puede descargarse y comenzar a utilizarse.

Sitio web oficial: <https://start.spring.io/>

4. ¿Cuáles son las anotaciones para identificar un controlador, servicio y repositorio?

@Controller

Identifica una clase como controlador MVC. El controlador recibe las solicitudes del usuario, las procesa y determina qué respuesta o vista debe devolver.

@Service

Identifica una clase como servicio. El servicio contiene principalmente la lógica de negocio de la aplicación y se encuentra entre el controlador y el acceso a datos.

@Repository

Identifica un componente encargado del acceso a datos. Su función es comunicarse con la base de datos y realizar operaciones de persistencia. En Spring Data es habitual utilizar JpaRepository.

La relación entre las capas puede representarse como:

Controlador → Servicio → Repositorio → Base de datos

El Controlador recibe las solicitudes, el Servicio aplica la lógica de negocio y el Repositorio realiza el acceso a los datos.

5. Diferencia entre el patrón DAO y Repository de Spring Boot

DAO (Data Access Object) es un patrón que proporciona una abstracción para acceder a los datos. Una implementación DAO puede utilizar directamente tecnologías como JDBC o SQL y normalmente requiere implementar los métodos de acceso.

Repository también abstrae el acceso a datos, pero representa conceptualmente una colección de objetos del dominio. Con Spring Data, gran parte de la implementación puede ser generada automáticamente.

DAO:

- Se centra en el acceso a datos.
- Puede requerir código manual.
- Puede utilizar JDBC directamente.
- Requiere implementar operaciones.

Repository:

- Se centra en la persistencia de objetos del dominio.
- Spring Data automatiza gran parte del código.
- Normalmente utiliza JPA/Spring Data.
- Proporciona operaciones CRUD automáticamente.

Ambos buscan separar el acceso a datos del resto de la aplicación, pero Repository de Spring permite trabajar con una abstracción de mayor nivel y reduce el código necesario.

6. ¿Para qué sirve el archivo pom.xml? ¿Qué formato tiene y qué puede contener?

El archivo pom.xml es el archivo principal de configuración de un proyecto que utiliza Maven. POM significa Project Object Model.

Su formato es XML.

Puede contener:

- Nombre e identificación del proyecto.
- groupId.
- artifactId.
- version.
- Dependencias.
- Plugins de Maven.
- Configuración de compilación.
- Versión de Java.

- Configuración de Spring Boot.
- Configuración para realizar pruebas y empaquetar la aplicación.

Las dependencias se agregan mediante elementos como `<dependency>`. Por ejemplo, un proyecto Spring Boot puede tener dependencias para Spring Web, Spring Data JPA, MySQL, Thymeleaf o testing.

7. Explicar el patrón Inyección de Dependencia. ¿Cómo se aplica en Spring Boot?

La Inyección de Dependencia (Dependency Injection) es un patrón mediante el cual una clase recibe los objetos que necesita desde el exterior, en lugar de crearlos directamente.

Sin inyección de dependencias, una clase podría crear directamente el objeto que necesita mediante `new`.

Con Spring, por ejemplo:

```
@Service
public class PaisService {

    @Autowired
    private PaisRepository repository;
}
```

Spring se encarga de crear y proporcionar la instancia de `PaisRepository`.

Spring utiliza un contenedor de Inversión de Control (IoC) que administra los objetos de la aplicación. Las clases identificadas con anotaciones como `@Controller`, `@Service` y `@Repository` son administradas por Spring y pueden ser inyectadas como dependencias.

La inyección de dependencias permite:

- Reducir el acoplamiento entre clases.
- Facilitar el mantenimiento.
- Facilitar las pruebas.
- Reutilizar componentes.
- Evitar crear manualmente las dependencias.